МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по практической работе №2
по дисциплине «Интеллектуальные системы анализа геопространственных данных»
Тема: Расчет спектральных индексов и кластеризация зон загрязнения

Студент гр. 1304 _____ Гирман А.В.

Преподаватель _____ Уткина О.Н.

Санкт-Петербург
2026

Цель работы.

Освоение практических навыков расчета спектральных индексов по спутниковым данным и применения методов кластеризации для выявления зон экологического загрязнения.

Задачи:

1. Освоить методы расчета спектральных индексов (NDVI, NDWI, NDBI) по снимкам Sentinel-2.
2. Научиться интерпретировать полученные индексы для оценки состояния экосистем.
3. Освоить метод кластеризации DBSCAN для выявления зон загрязнения по точечным пробам.
4. Научиться визуализировать результаты на интерактивных картах.
5. Продолжить развитие класса WaterQualityPipeline для сквозного проекта.
6. Подготовить данные для последующего машинного обучения.

Задание варианта 3.

Регион: Астраханская область

Водный объект: Дельта реки Волги

Координаты (примерные границы): 45.0–46.0° с.ш., 47.0–49.0° в.д.

Теоретический минимум.

Спектральные индексы – это комбинации значений в разных спектральных каналах, которые позволяют выделить интересующие объекты или оценить их состояние.

Формула **NDVI (Normalized Difference Vegetation Index)**

$$NDVI = (NIR - RED) / (NIR + RED)$$

где каналы Sentinel-2:

- RED: B4 (665 нм) – красный канал;

- NIR: B8 (842 нм) – ближний инфракрасный.

Интерпретация о состоянии растительности:

- NDVI < 0 – Вода, снег, облака, открытые почвы;
- 0 < NDVI < 0.2 – Открытая почва, разреженная растительность;
- 0.2 < NDVI < 0.5 – Умеренная растительность;
- 0.5 < NDVI < 0.8 – Густая растительность;
- NDVI > 0.8 – Очень густая растительность (тропические леса);

Формула **NDWI (Normalized Difference Water Index)**:

$$NDWI = (GREEN - NIR) / (GREEN + NIR)$$

где каналы Sentinel-2:

- GREEN: B3 (560 нм) – зеленый канал;
- NIR: B8 (842 нм) – ближний инфракрасный.

Интерпретация о состоянии объекта:

- NDWI > 0 – Водные объекты;
- NDWI < 0 – Суша, растительность;

Формула **NDBI (Normalized Difference Built-up Index)**:

$$NDBI = (SWIR - NIR) / (SWIR + NIR)$$

где каналы Sentinel-2:

- SWIR: B11 (1610 нм) – коротковолновый инфракрасный;
- NIR: B8 (842 нм) – ближний инфракрасный.

Интерпретация о состоянии объекта:

- NDWI > 0 – Застроенные территории;
- NDWI < 0 – Растительность, вода;

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) – алгоритм кластеризации, основанный на плотности точек.

Преимущества DBSCAN:

- не требует задания количества кластеров;
- находит кластеры произвольной формы;
- устойчив к выбросам (шум помечается отдельно);
- хорошо работает с пространственными данными;
- чувствителен к выбору параметров Eps и MinPts, особенно в данных с сильно различающейся плотностью

Спутниковая программа Sentinel-2

Характеристики Sentinel-2:

- пространственное разрешение: 10 м (B2, B3, B4, B8), 20 м (B5-B7, B8A, B11, B12), 60 м (B1, B9, B10);
- временное разрешение: 5 дней (на экваторе);
- спектральных каналов: 13;
- ширина полосы захвата: 290 км.

STL (Seasonal-Trend decomposition using LOESS) — метод декомпозиции временного ряда на компоненты:

- тренд — долгосрочная тенденция;
- сезонность — периодические колебания;
- остатки — случайные отклонения (аномалии).

Экологическое применение: выявление аномальных лет по NDVI (засухи, болезни леса, вырубки).

Выполнение работы.

1. Расширение класса WaterQualityPipeline.

1.1 Создан класс *WaterQualityPipelineExtended*. Он наследуется от класса *WaterQualityPipeline* из практической работы №1. Класс имеет 5 полей:

- raster – хранит открытый растровый снимок Sentinel-2 вместе с метаданными;
- bands – хранит трёхмерный массив значений всех спектральных каналов снимка размером;
- indices – словарь рассчитанных спектральных индексов;
- clusters – хранит номера кластеров, которые DBSCAN присвоил точкам проб;
- points_gdf – хранит GeoDataFrame с точками отбора проб: координаты, концентрации NH4 и PO4, pH, участок и номер кластера.

1.2 Добавлен метод *load_raster*. Метод открывает tiff-файл через *rasterio* и сохраняет объект снимка в атрибут *raster*. Затем он читает все каналы в массив NumPy и сохраняет его в атрибут bands. После загрузки метод выводит в консоль размер снимка, число каналов и систему координат (см. рис. 1).

```
Создан объект: Практическая работа №2
Снимок загружен: data/delta_volga.tif
Размер: 1556 x 1683
Каналов: 5
CRS: EPSG:32639
```

Рисунок 1 – Вывод данных о снимке

2.1 Добавлены методы *calculate_**. Каждый метод вычисляет свой индекс: NDVI, NDWI, NDBI. Результат сохраняется в словарь *indices*. Метод *calculate_all_indices* вызывает все три метода по очереди. Снимки индексов приведены на рисунках 2 – 4.

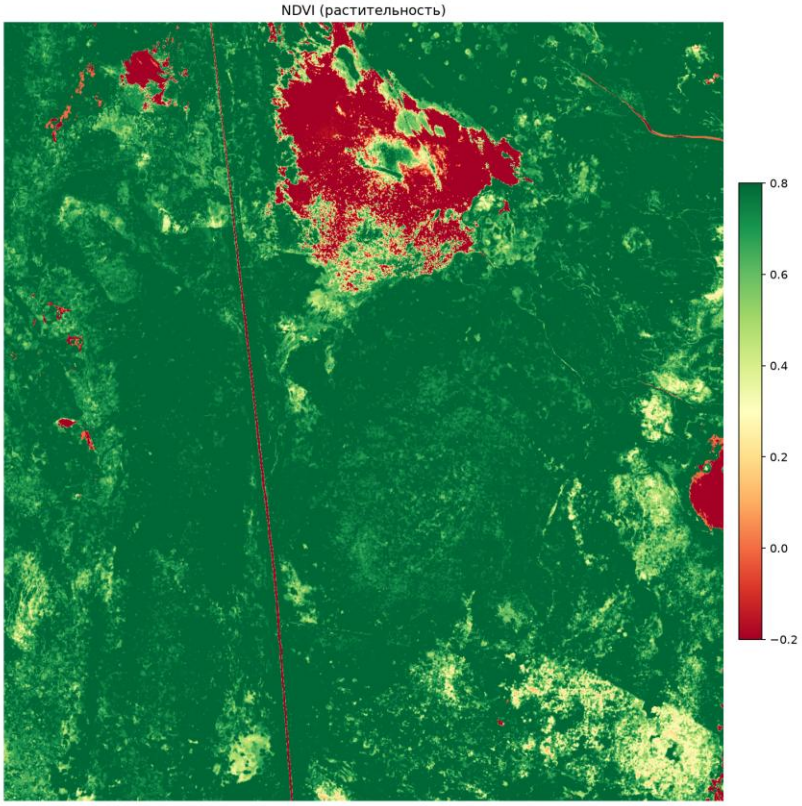


Рисунок 2 – Значение индекса NDVI на снимке

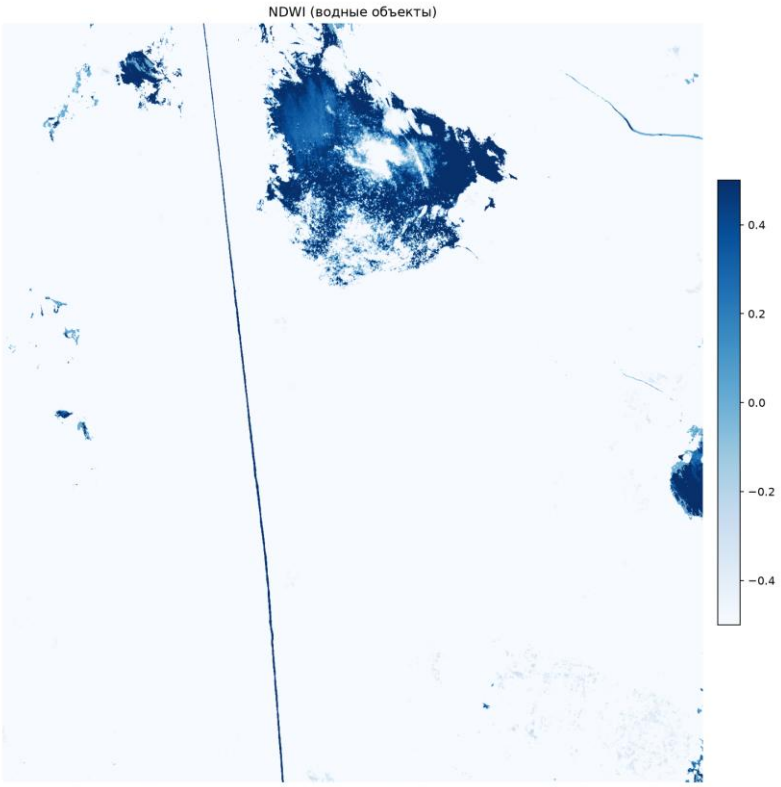


Рисунок 3 – Значение индекса NDWI на снимке

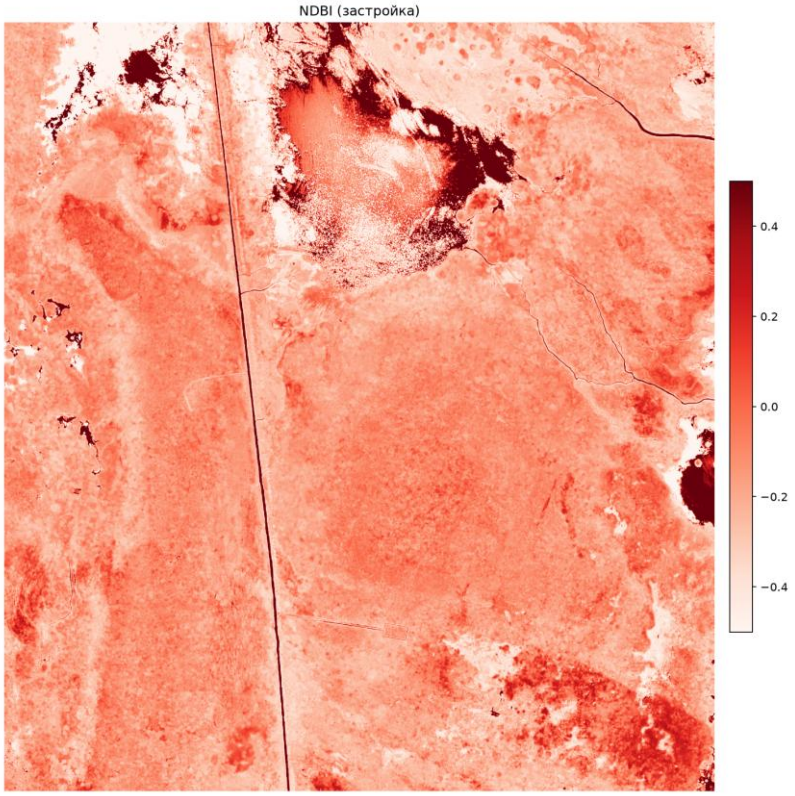


Рисунок 4 – Значение индекса NDBI на снимке

Из рисунков 2 – 4 следует, что значения NDVI на большей части снимка достигают 0.8, что свидетельствует о густом растительном покрове на данной территории. Также имеются участки с отрицательными значениями NDVI и при сопоставлении с NDWI показывает, что на этих же участках индекс принимает положительные значения, что позволяет отнести их к водным поверхностям. Значения NDBI почти на всей площади отрицательны, а области его повышенных значений совпадают с пониженными значениями NDVI. Это даёт основание интерпретировать такие участки как сухую или разреженную растительность, а не как застройку.

3. Визуализация индексов с точками проб.

3.1 Добавлен метод *create_water_sample_points*. Он создаёт тестовые точки отбора проб воды с гидрохимическими показателями: NH4 (аммоний), PO4 (фосфаты) и pH (водородный показатель). Точки создаются в границах загруженного снимка и в его системе координат (EPSG:32639). Метод делит 50

точек на 4 группы, у каждой группы свои диапазоны показателей (см. таблица 1).

Таблица 1 – Параметры групп проб

Группа	NH4, мг/л	PO4, мг/л	pH
чистая вода	0.05 – 0.25	0.02 – 0.10	7.8 – 8.3
органическое загрязнение	1.2 – 2.2	0.25 – 0.55	6.8 – 7.4
аммонийное загрязнение	3.5 – 5.8	0.60 –1.10	6.3 – 6.9
промышленное загрязнение	7.0 – 13.0	1.50 – 3.20	5.4 – 6.3

3.2 Добавлен метод *plot_with_points*, который строит шесть графиков подряд, каждый на отдельном полотне:

- *plot_rgb_with_points* – снимок в естественных цветах (каналы B4, B3, B2) с точками проб. Перед отображением яркость каждого канала растягивается по 2–98 перцентилям, иначе 16-битный снимок выглядит слишком тёмным. Цвет точки зависит от концентрации NH4: зелёный до 0.5 мг/л, оранжевый от 0.5 до 1.0 мг/л, красный больше 1.0 мг/л.
- *plot_index* – карта одного спектрального индекса (NDVI, NDWI и NDBI) со шкалой значений.
- *plot_nh4_histogram* – гистограмма концентрации NH4 с линией среднего значения.
- *plot_locations* – число проб на каждом участке.

Визуализацию точек проб на снимке, можно увидеть на рисунке 5, а распределения по участкам на рисунке 7.

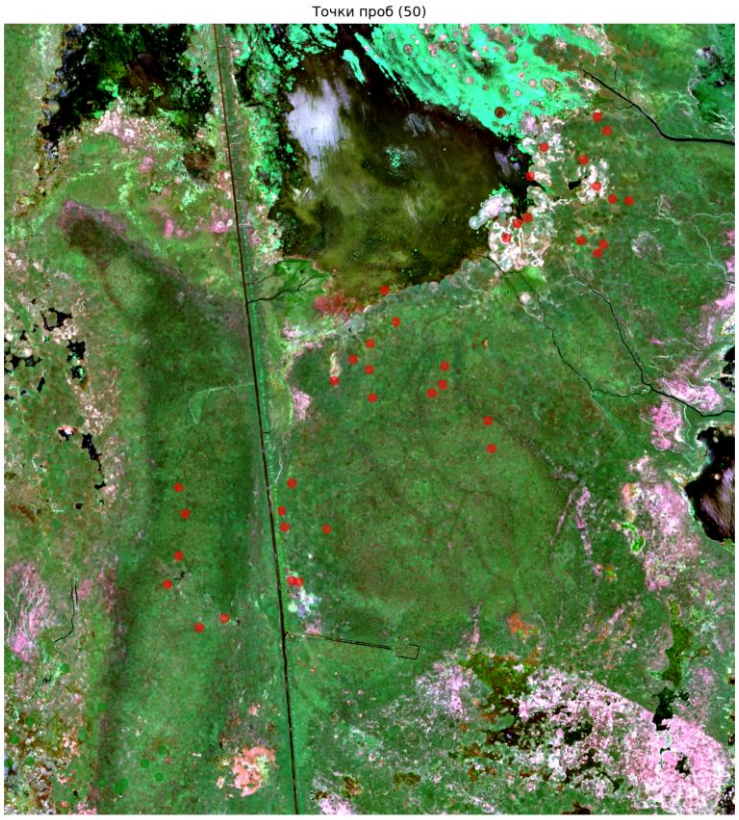


Рисунок 5 – Визуализация точек проб

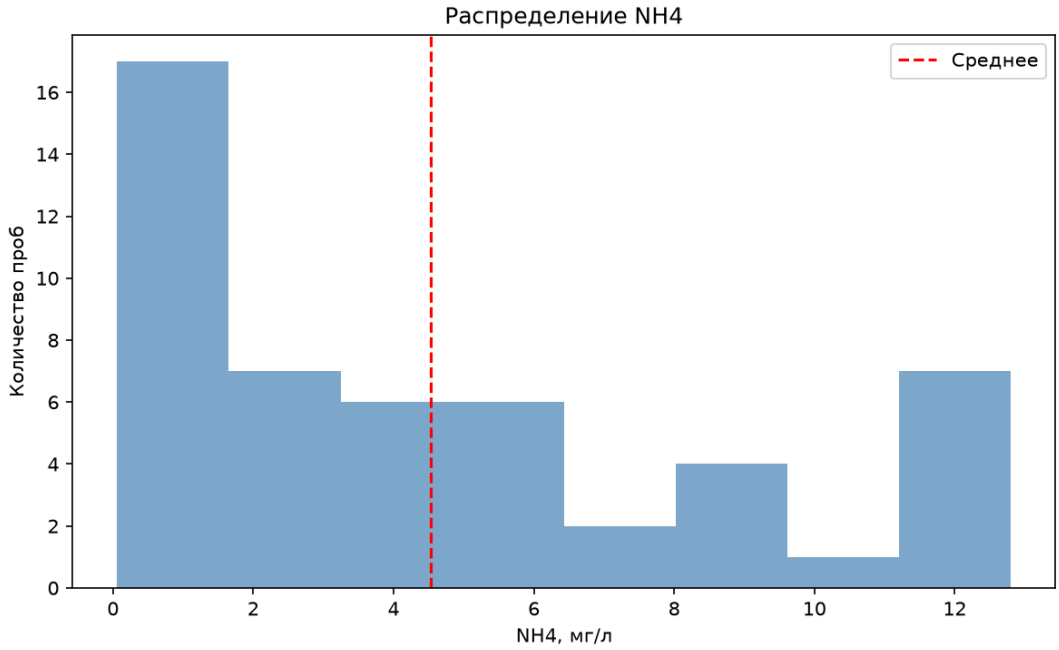


Рисунок 6 – Гистограмма распределения NH4 (аммоний)

Из рисунка 6 следует, что концентрации NH4 в пробах варьируют от 0 до 12 мг/л и выше при среднем значении около 4.5 мг/л. Большинство проб

характеризуется низкими концентрациями, однако выделяется группа проб с высокими значениями выше 12 мг/л.

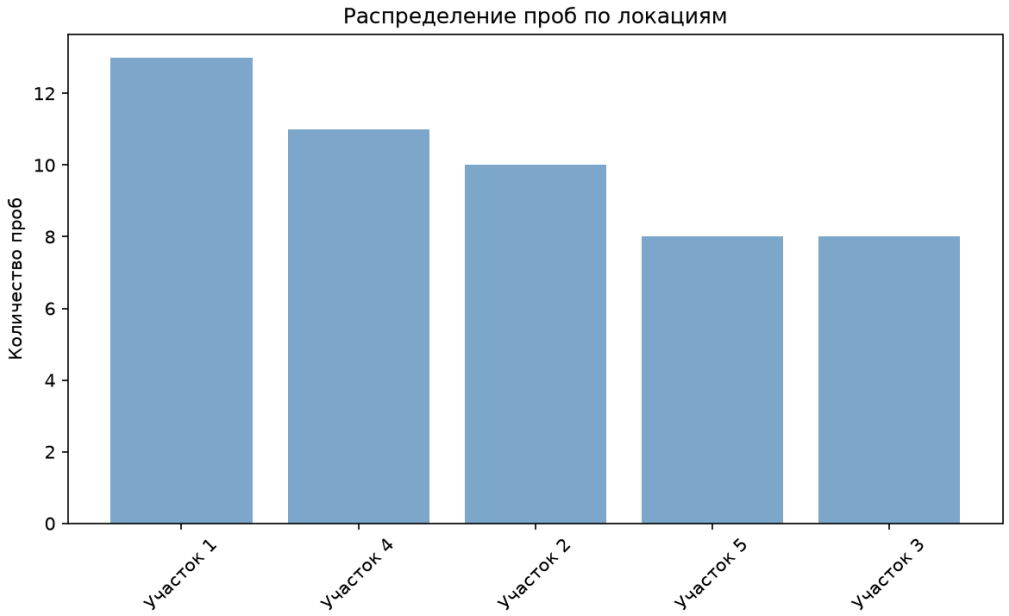


Рисунок 7 – Гистограммы распределения проб по локациям

4. Кластеризация зон загрязнения.

4.1 Для поиска зон загрязнения используется алгоритм DBSCAN. Он объединяет в кластер точки близкие друг к другу по трём гидрохимическим показателям: NH4, PO4 и pH. Точки, у которых мало соседей по показателям, алгоритм считает шумом и ставит им метку -1.

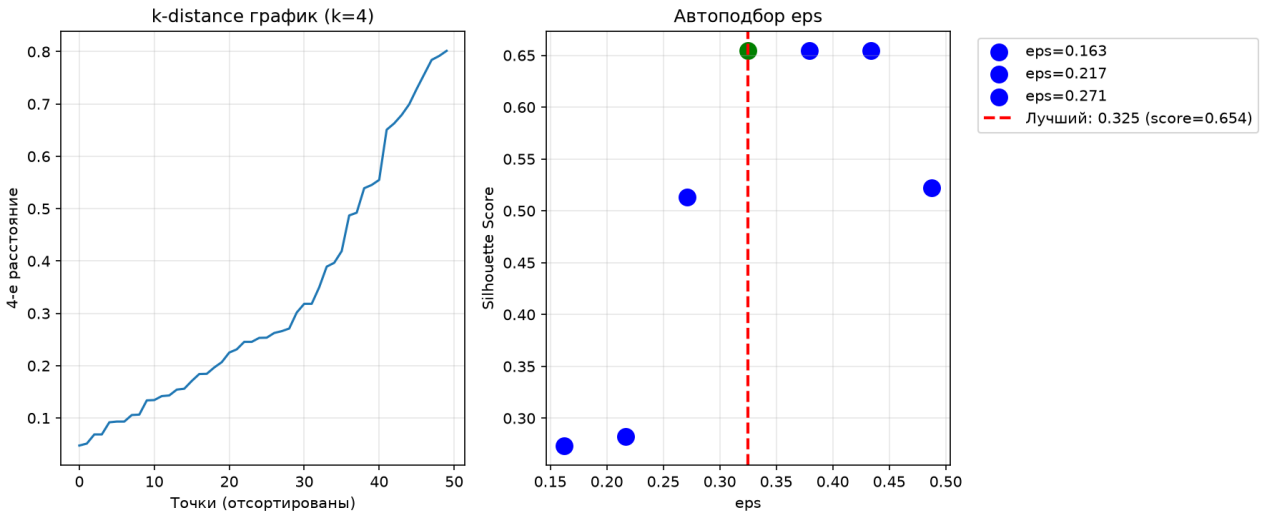


Рисунок 8 – Графики изменения k-distance и подбор лучшего eps

На графиках (см. рис. 8) показаны отсортированные расстояния от каждой точки до её 4-го ближайшего соседа. Примерно у 30 точек расстояние растёт плавно, до 0.3-0.4. Дальше график резко поднимается до 0.8, это означает, что точки расположены редко с сильным разбросом значений. Подходящее значение *eps* лежит в области перегиба, найденное 0.325 соответствует этой области.

5. Визуализация кластеров на карте.

Для отображения результатов кластеризации добавлен метод *visualize_clusters_map*. Он строит интерактивную карту *Folium*, на которой каждая точка проб окрашена в цвет своего кластера (см. рис. 9).

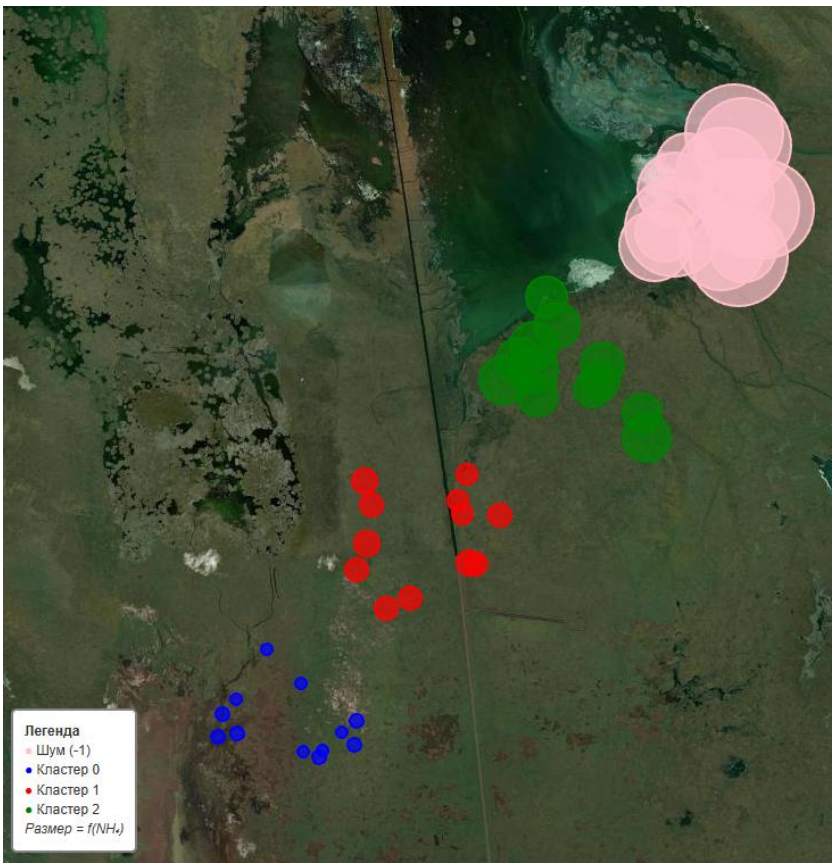


Рисунок 9 – Карта кластеров зон загрязнения

На рисунке 9, каждая проба показана на карте кружком с разным радиусом и цветом. Цвет определяет принадлежность пробы к определённому кластеру, а радиус зависит от концентрации NH4.

6. Анализ временных рядов NDVI и выявление аномалий.

6.1 Для анализа динамики растительности добавлен метод *analyze_ndvi_anomalies*. Он создаёт синтетический ряд NDVI. Ряд содержит 120 месячных значений за 2015-2024 годы и складывается из четырёх частей:

- сезонность – зимой (0.25-0.28), летом (0.70-0.78);
- тренд – постепенное снижение на 0.12 за 10 лет (имитирует медленную деградацию растительности);
- шум – случайные отклонения с нормальным распределением;

- аномалии – в двух случайных годах снижается на 0.15-0.30 на срок от 4 до 8 месяцев, начиная с мая до сентября. Так имитируются засуха или вырубка.

Созданы также методы рисования графиков *plot_stl_component*, *plot_stl_residuals*, *plot_ndvi_with_anomalies*. Данный функции нужны для последующего анализа рисунков 10-14.

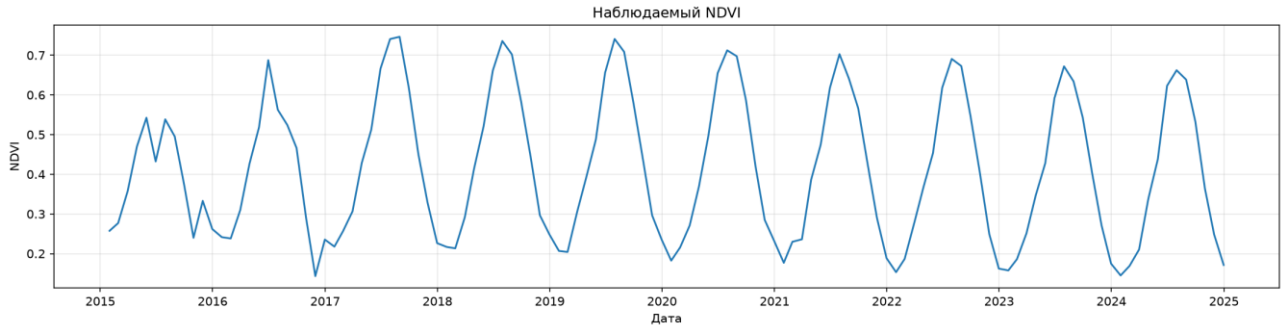


Рисунок 10 – Наблюдаемый ряд NDVI

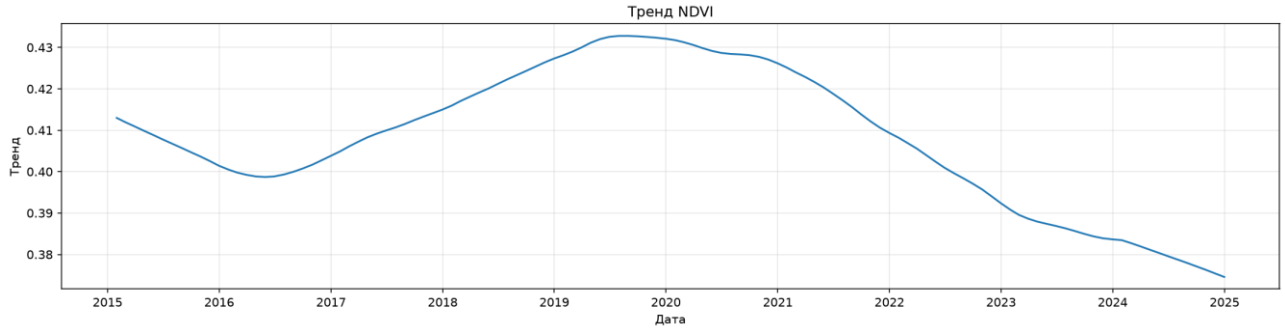


Рисунок 11 – Тренд NDVI

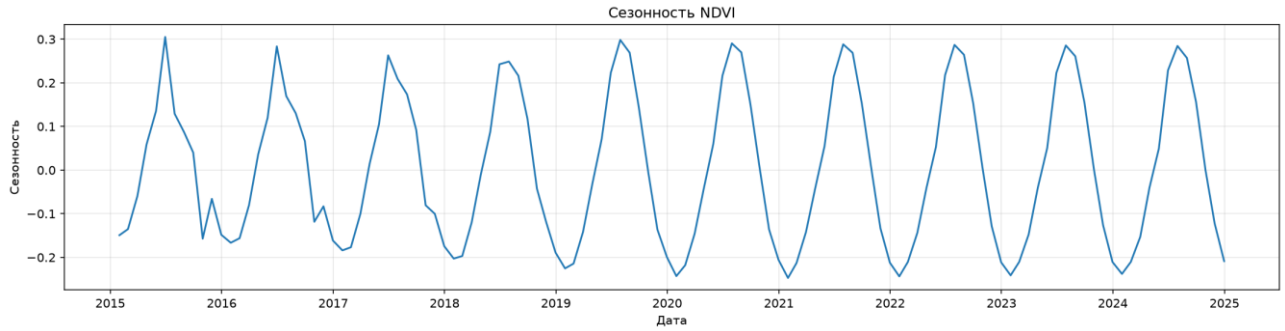


Рисунок 12 – Сезонность NDVI



Рисунок 13 – Остатки STL-разложения с коридором 2σ



Рисунок 14 – Ряд NDVI с найденными аномалиями

Из рисунков 10-14, можно сделать вывод, что ряд NDVI за промежуток 2015-2024 года имеет чёткий годовой цикл около 0.2 зимой и 0.7 летом. STL-разложение показало сезонную волну с размахом около 0.53 и снижение тренда с 0.43 до 0.37, то есть деградацию растительности. Остатки вышли за коридор 2σ вниз в 2015, 2016 годах и больше всего было замечено аномалий за промежуток 2017-2018 года.

Выполнение самостоятельных заданий.

*** 2.3 Для решения поставленной задачи было скачано два снимка одной местности с промежутком по времени с 01.06.2025 по 01.08.2025. Полученные снимки с индексом NDVI можно увидеть на рисунках 15 и 16.

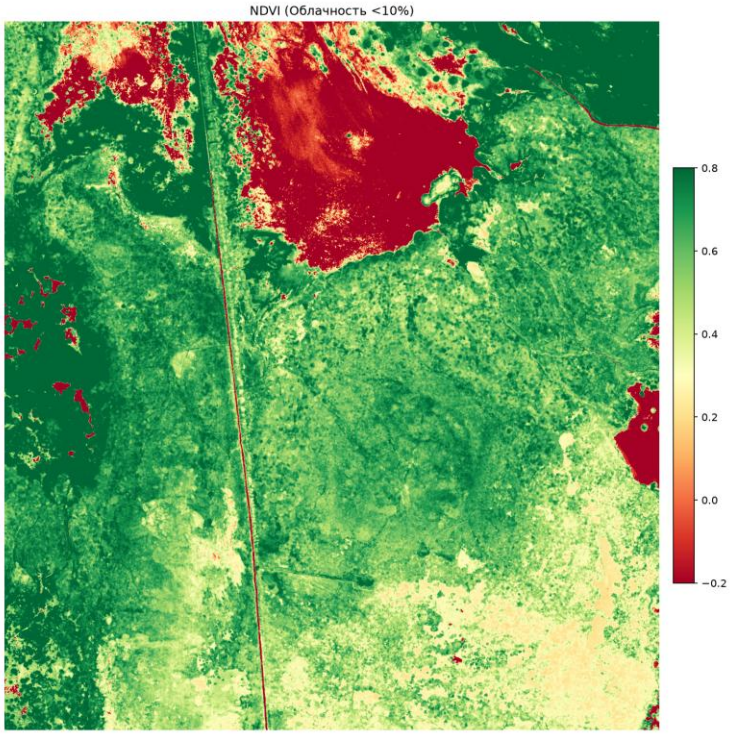


Рисунок 15 – Значение индекса NDVI на снимке с облачностью <10%

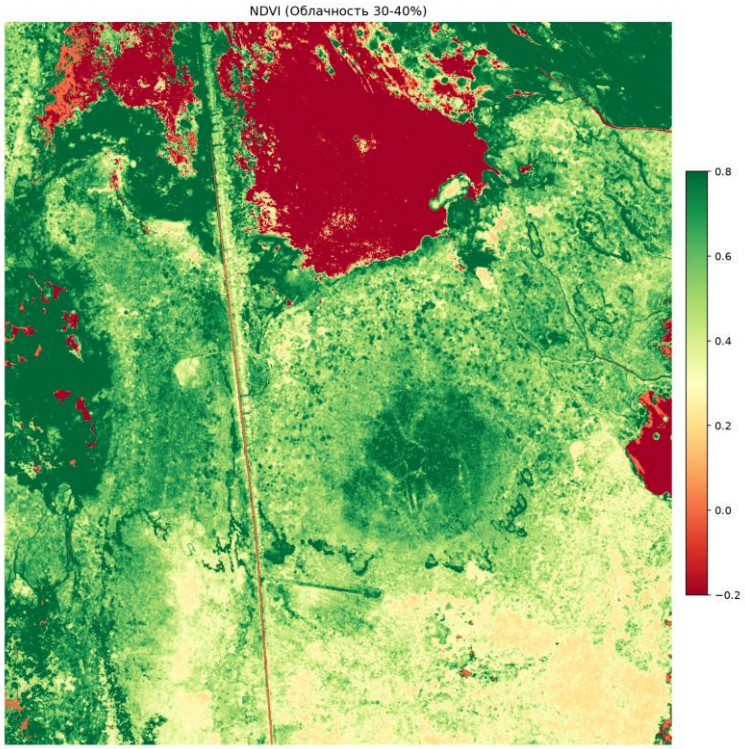


Рисунок 16 – Значение индекса NDVI на снимке с облачностью 30-40%

Рассматривая рисунки 15 и 16, можно прийти к выводу, что при облачности менее 10% растительность имеет NDVI около 0.5-0.7 (больше

зеленных участков), а при облачности 30-40% на тех же участках значения снижаются до 0.3-0.5 (участки стали иметь желтый оттенок). Кроме того, на втором снимке шире зоны с низкими значениями на юго-востоке и встречаются локальные артефакты. Это объясняется тем, что облака или дымка смещают NDVI к нулю.

Для более подробного анализа значений NDVI была составлена таблица (см. таблицу 2) и гистограммы распределения (см. рис. 17).

Таблица 2 – Статистика NDVI для двух снимков

Снимок	Среднее значение	Медиана	СКО
Облачность <10%	0.49903	0.60144	0.36581
Облачность 30-40%	0.43190	0.53640	0.43366

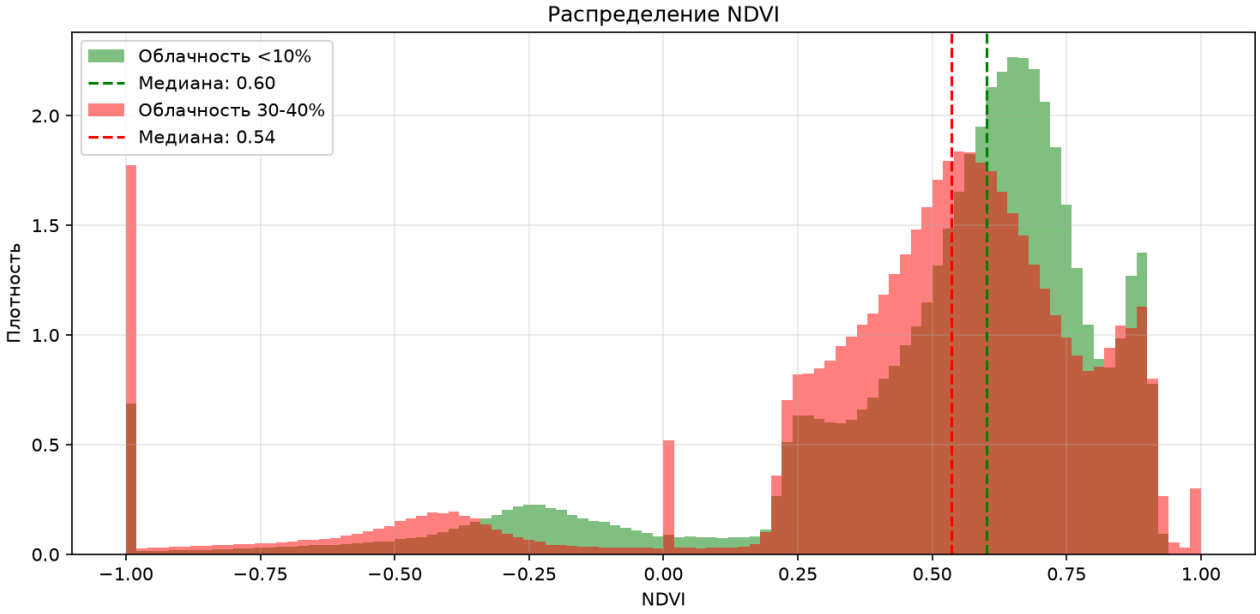


Рисунок 17 – Гистограмма распределения NDVI с малой и средней облачностью

По данным из таблицы 2 и рисунку 17 видно, что при облачности 30-40% все показатели центральной тенденции ниже, чем при облачности менее 10%. Среднее значение NDVI снизилось примерно на 0.067, медиана снизилась с 0.601

до 0.536. При этом СКО выросло, что значит при повышенной облачности разброс значений стал больше, что и можно пронаблюдать на гистограмме.

Для более подробного анализа был взят снимок с облачностью больше 40% (с. рис. 18).

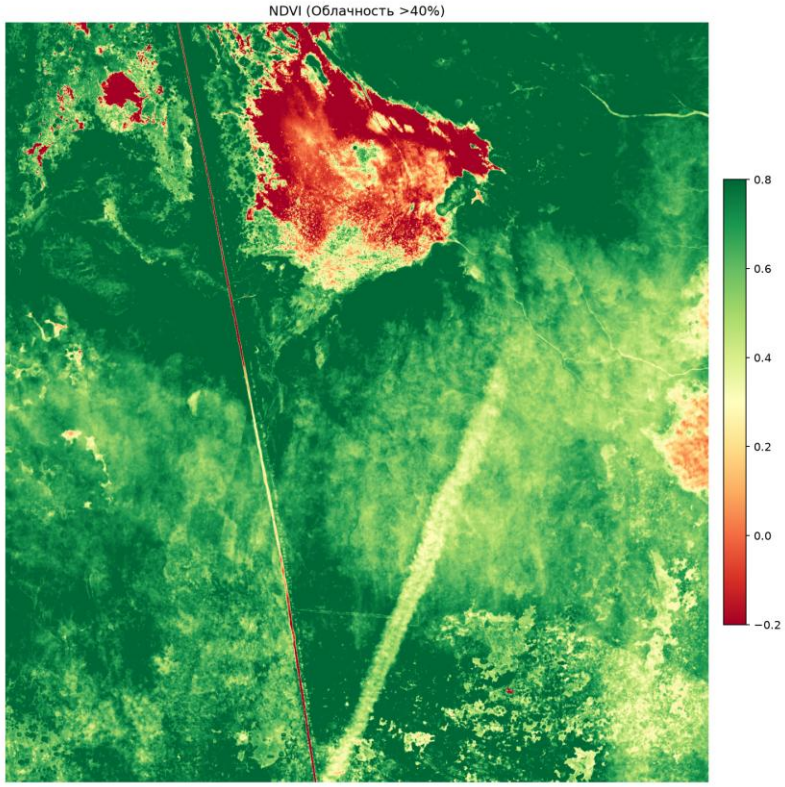


Рисунок 18 – Значение индекса NDVI на снимке с облачностью >40%

Рассматривая рисунки 18, можно прийти к выводу, что через центральную часть по диагонали проходит светлая полоса облака, а в центре и на востоке видны размытые участки. Сильнее всего искажены мелкие водные объекты, которых стало практически не различить. При этом значительная часть растительности имеет высокие значения NDVI примерно 0.6-0.8. Таким образом, при облачности более 40% участки водоемов сильно искажены и их невозможно корректно интерпретировать.

Для более подробного анализа значений NDVI была составлена таблица (см. таблицу 3) и гистограммы распределения (см. рис. 19).

Таблица 3 – Статистика NDVI для двух снимков

Снимок	Среднее значение	Медиана	СКО
Облачность <10%	0.49903	0.60144	0.36581
Облачность >40%	0.62866	0.68392	0.25618

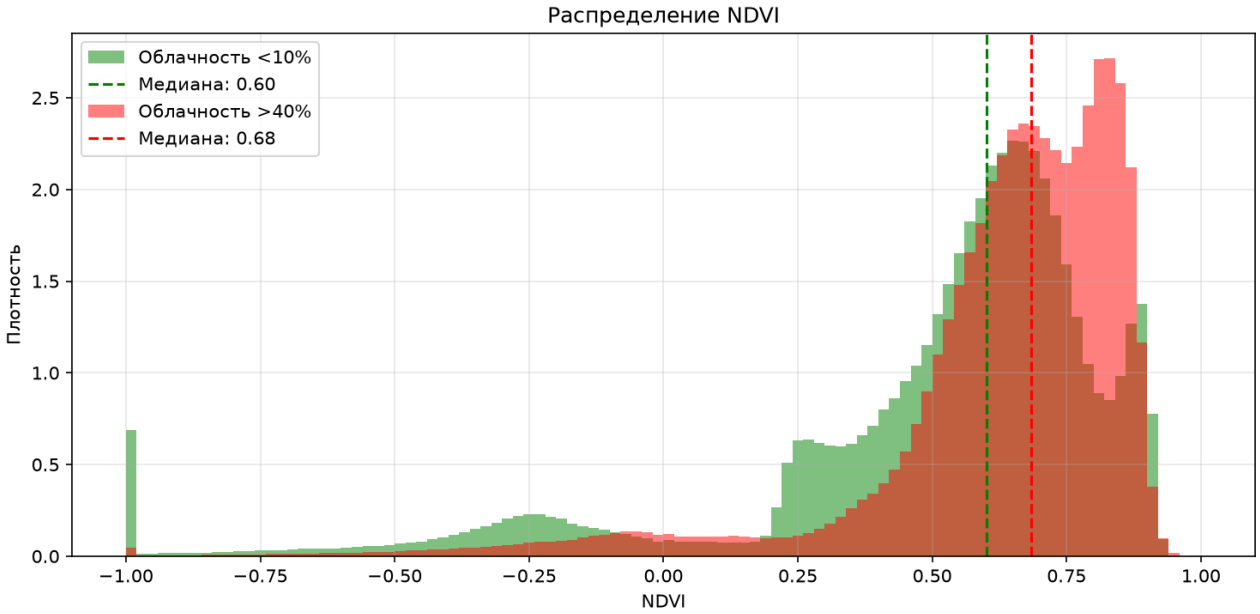


Рисунок 19 – Гистограмма распределения NDVI с малой и большой облачностью

По данным таблицы 3 и рисунку 19, можно сказать, что снимок с облачностью более 40%, вопреки ожиданиям, показывает более высокие значения NDVI, чем малооблачный снимок. Среднее значение выросло, медиана выросла, а СКО снизилось. Гистограмма объясняет этот результат тем, что на снимке с облачностью более 40% почти отсутствуют отрицательные значения. В районах, соответствующих водным объектам почти нет значения -0.25, и нет выброса на -1. Рост средних значений и снижение разброса связаны в первую очередь с тем, что облака над водоёмами повысили их NDVI и исключили отрицательные значения из распределения.

Таким образом, результаты анализа показывают, что надёжные значения NDVI получены только для снимка с облачностью менее 10%. Уже при

облачности 30-40% искажения становятся значимыми, а при облачности более 40% искажения настолько сильны, что статистические показатели меняются в противоположную сторону, а водные объекты частично теряются. Следовательно, результаты становятся ненадёжными уже при облачности около 30% и выше.

* 5.1 Для решения поставленной задачи был изменен метод *visualize_clusters_map*, а также добавлены методы:

- *most_polluted_cluster* – дынный метод находит кластер у которого средняя концентрация NH4 наибольшая. Точки шума не учитываются.
- *ndvi_near_water* – данный метод возвращает NDVI в пикселе, где стоит точка, и проверяет, есть ли в заданном радиусе вода (по NDWI).
- *get_recommendations* – составляет список рекомендаций для точки: «Рекомендуется отбор повторной пробы» (проверяется по методу *most_polluted_cluster*), «Проверить координаты: возможна ошибка GPS» (проверяется по номеру кластера), «Возможен сброс сточных вод» (проверяется по методу *ndvi_near_water*).

На рисунках 20-22, можно увидеть полученные рекомендации.

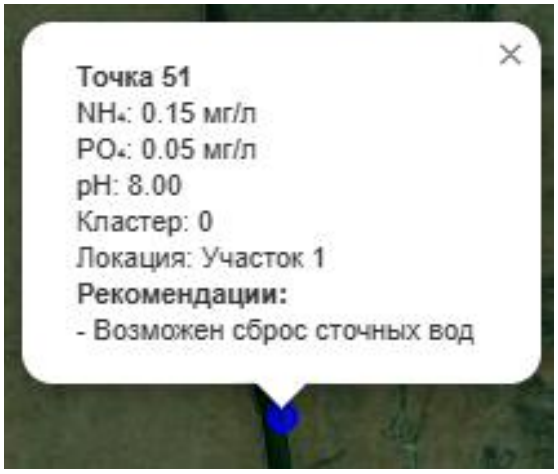


Рисунок 20 – Рекомендация «Возможен сброс сточных вод»



Рисунок 21 – Рекомендация «Рекомендуется отбор повторной пробы»



Рисунок 22 – Рекомендация «Проверить координаты: возможна ошибка GPS»

Такие «умные подсказки» (см. рис. 20-22) могут напрямую ускорить работу полевого инспектора, так как система сама анализирует данные и сразу показывает, какие точки требуют внимания и какие действия нужны. Благодаря этому инспектор быстрее планирует маршрут и реже выезжает повторно.

Выводы.

Освоены практические навыки расчёта спектральных индексов по снимкам Sentinel-2 и применения кластеризации для выявления зон загрязнения. Реализован класс *WaterQualityPipelineExtended* с методами расчёта NDVI, NDWI и NDBI, кластеризации проб методом DBSCAN, визуализации на карте Folium и анализа временных рядов NDVI. Анализ индексов показал преобладание густой растительности на территории дельты Волги и позволил выделить водные

объекты. DBSCAN с автоматически подобранным *eps* разделил пробы на кластеры по уровню загрязнения. STL-разложение выявило снижение тренда NDVI и аномальные периоды. Эксперимент со снимками разной облачности показал, что облака искажают NDVI как в меньшую, так и в большую сторону, а надёжные результаты получаются только при облачности менее 10%. Добавленные «умные подсказки» помогают инспектору быстрее определять приоритетные точки для проверки.

ПРИЛОЖЕНИЕ А

Исходный код программы

Файл main.py:

```
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler
from water_quality_pipeline_extended import WaterQualityPipelineExtended
import geopandas as gpd
import numpy as np
import os


def task_1_2(pipeline: WaterQualityPipelineExtended):
    path_to_file = 'data/delta_volga.tif'
    pipeline.load_raster(path_to_file)


def task_2_1(pipeline: WaterQualityPipelineExtended):
    indices = pipeline.calculate_all_indices()
    for name, values in indices.items():
        print(f'{name}: min = {values.min():.3f}, max = {values.max():.3f}')


def task_3_1(pipeline: WaterQualityPipelineExtended):
    pipeline.create_water_sample_points(n_points=50)
    pipeline.points_gdf = gpd.read_file('data/water_points.geojson')


def task_3_2(pipeline: WaterQualityPipelineExtended):
    pipeline.plot_with_points()


def task_4_1(pipeline: WaterQualityPipelineExtended):
```

```
        if (pipeline.points_gdf is None) or
len(pipeline.points_gdf) == 0:
            raise ValueError("points_gdf должен быть заполнен
данными")

        features = ['NH4_mgL', 'PO4_mgL', 'pH']
        X = pipeline.points_gdf[features].values
        X_scaled = StandardScaler().fit_transform(X)
        optimal_eps = pipeline.find_optimal_eps_auto(X_scaled,
min_samples=4)
        pipeline.clusters = DBSCAN(eps=optimal_eps,
min_samples=4).fit_predict(X_scaled)
        pipeline.points_gdf['cluster'] = pipeline.clusters
        n_clusters = len(set(pipeline.clusters)) - (1 if -1 in
pipeline.clusters else 0)
        print(f"Найдено {n_clusters} кластеров
(eps={optimal_eps:.3f})")


    def task_5(pipeline: WaterQualityPipelineExtended):

pipeline.visualize_clusters_map(filename='class_data/clusters_map.
html')
        os.makedirs('class_data', exist_ok=True)

pipeline.points_gdf.to_file('class_data/points_with_clusters.geojs
on', driver='GeoJSON')
        print("\nТочки с кластерами сохранены:
class_data/points_with_clusters.geojson")


    def task_6_1(pipeline: WaterQualityPipelineExtended):
        pipeline.analyze_ndvi_anomalies(n_years=10, seed=42)
        print("\nИТОГОВАЯ СТАТИСТИКА")
        print("="*50)
```

```
        print(f"Всего точек проб: {len(pipeline.points_gdf)}")
        print(f"Средний                                    NDVI:
{pipeline.indices['NDVI'].mean():.3f}")
        print(f"Средний                                    NDWI:
{pipeline.indices['NDWI'].mean():.3f}")
        print(f"Средний                                    NDBI:
{pipeline.indices['NDBI'].mean():.3f}")
        print(f"Диапазон                                    NH4:
{pipeline.points_gdf['NH4_mgL'].min():.2f}                          -
{pipeline.points_gdf['NH4_mgL'].max():.2f} мг/л")


    def          task_2_3(clear_path='data/delta_volga_lt_10.tif',
cloudy_path='data/delta_volga_gte_30_lte_40.tif'):
        name_1 = 'Облачность <10%'
        name_2 = 'Облачность 30-40%'
        # name_3 = 'Облачность >40%'

        clear = WaterQualityPipelineExtended(name_1)
        clear.load_raster(clear_path)
        ndvi_clear = clear.calculate_ndvi()
        clear.plot_index('NDVI',  'RdYlGn',  -0.2,  0.8,  f'NDVI
({name_1})', 'ndvi_lt_10.png')

        cloudy = WaterQualityPipelineExtended(name_2)
        cloudy.load_raster(cloudy_path)
        ndvi_cloudy = cloudy.calculate_ndvi()
        cloudy.plot_index('NDVI',  'RdYlGn',  -0.2,  0.8,  f'NDVI
({name_2})', 'ndvi_gte_30_lte_40.png')

        clear.plot_ndvi_histograms({
            name_1: ndvi_clear,
            name_2: ndvi_cloudy,
        }, 'ndvi_cloud_hist.png')
```

```
print(f"\n{'Снимок':<20}{'Среднее':>10}{'Медиана':>10}{'СКО':>10}"
)
        for label, ndvi in [(name_1, ndvi_clear), (name_2,
ndvi_cloudy)]:
            mean = round(float(ndvi.mean()), 5)
            median = round(float(np.median(ndvi)), 5)
            std = round(float(ndvi.std()), 5)
            print(f"{label:<20}{mean:>10}{median:>10}{std:>10}")


    def main():
        pipeline = WaterQualityPipelineExtended()
        task_1_2(pipeline)
        task_2_1(pipeline)
        task_3_1(pipeline)
        task_3_2(pipeline)
        task_4_1(pipeline)
        task_5(pipeline)
        task_6_1(pipeline)

        # task_2_3()

    if __name__ == '__main__':
        main()
```

Файл water_quality_pipeline_extended.py:

```
from water_quality_pipeline import WaterQualityPipeline
import rasterio
from rasterio.io import DatasetReader
import geopandas as gpd
import matplotlib.pyplot as plt
import numpy as np
import os
from sklearn.cluster import DBSCAN
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors
import folium
import webbrowser
from kneed import KneeLocator
from sklearn.metrics import silhouette_score
import pandas as pd
from statsmodels.tsa.seasonal import STL
import random


class WaterQualityPipelineExtended(WaterQualityPipeline):
    def __init__(self, name="Практическая работа №2"):
        super().__init__(name)
        self.raster: DatasetReader | None = None
        self.bands: np.ndarray | None = None
        self.indices: dict[str, np.ndarray] = {}
        self.clusters: np.ndarray | None = None
        self.points_gdf: gpd.GeoDataFrame | None = None


    def load_raster(self, raster_path: str):
        """Загрузка спутникового снимка"""

        self.raster = rasterio.open(raster_path)

        if self.raster is None:
            raise FileExistsError("Неудалось открыть")

        self.bands = self.raster.read()

        print(f"Снимок загружен: {raster_path}")
        print(f"Размер:          {self.raster.width}          x
{self.raster.height}")
        print(f"Каналов: {self.raster.count}")
```

```
            print(f"CRS: {self.raster.crs}")

            return self.raster


        def calculate_ndvi(self, red_idx=0, nir_idx=3):
            """Вычисление NDVI"""

            if self.bands is None:
                raise ValueError("Сначала загрузите снимок
методомload_raster()")

            red = self.bands[red_idx].astype('float32')
            nir = self.bands[nir_idx].astype('float32')
            ndvi = (nir - red) / (nir + red + 1e-10)
            self.indices['NDVI'] = ndvi
            return ndvi


        def calculate_ndwi(self, green_idx=1, nir_idx=3):
            """Вычисление NDWI"""

            if self.bands is None:
                raise ValueError("Сначала загрузите снимок методом
load_raster()")

            green = self.bands[green_idx].astype('float32')
            nir = self.bands[nir_idx].astype('float32')
            ndwi = (green - nir) / (green + nir + 1e-10)
            self.indices['NDWI'] = ndwi
            return ndwi


        def calculate_ndbi(self, swir_idx=4, nir_idx=3):
            """Вычисление NDBI"""
```

```
            if self.bands is None:
             raise ValueError("Сначала загрузите снимок методом
load_raster()")

            swir = self.bands[swir_idx].astype('float32')
            nir = self.bands[nir_idx].astype('float32')
            ndbi = (swir - nir) / (swir + nir + 1e-10)
            self.indices['NDBI'] = ndbi
            return ndbi


        def calculate_all_indices(self):
            """Вычисление всех значений индексов"""

            self.calculate_ndvi()
            self.calculate_ndwi()
            self.calculate_ndbi()
            return self.indices


        def     create_water_sample_points(self,     n_points=50,
output_file='data/water_points.geojson'):
            """
            Создание точек проб с 4 заданными кластерами
            Алгоритм:
            1. Делит область снимка на 4 части по координатам
            2. В каждой части создает точки с разными диапазонами
NH4,
            PO4, pH
            3. Формирует 4 кластера для демонстрации DBSCAN
            Параметры кластеров (гауссовы распределения):
            - Кластер 0 (чистая вода): NH4=0.05-0.25, PO4=0.02-
0.10, pH=7.8-8.3
```

```
            - Кластер 1 (органика): NH4=1.2-2.2, PO4=0.25-0.55,
pH=6.8-7.4
            - Кластер 2 (аммоний): NH4=3.5-5.8, PO4=0.60-1.10,
pH=6.3-6.9
            - Кластер 3 (промышленное): NH4=7.0-13.0, PO4=1.50-
3.20, pH=5.4-6.3
            """

            if self.raster is None:
                raise ValueError("Сначала загрузите снимок методом
load_raster()")
            left, bottom, right, top = self.raster.bounds
            print(f"Границы снимка: X: {left:.1f}-{right:.1f}, Y:
{bottom:.1f}-{top:.1f}")

            np.random.seed(42)
            group_size = n_points // 4
            locations = locations = [
                'Участок 1',
                'Участок 2',
                'Участок 3',
                'Участок 4',
                'Участок 5'
            ]

            """
            Задаем 4 четких кластера - для учебных целей (гауссовы
распределения - сфера):
            увидеть работу алгоритма, поэкспериментировать с
параметрами,
            получить красивую карту с цветными точками
            """
            cluster_params = [
                {'NH4': (0.05, 0.25), 'PO4': (0.02, 0.10), 'pH':
(7.8, 8.3)},
```

```
                {'NH4': (1.2, 2.2), 'PO4': (0.25, 0.55), 'pH':
(6.8, 7.4)},
                {'NH4': (3.5, 5.8), 'PO4': (0.60, 1.10), 'pH':
(6.3, 6.9)},
                {'NH4': (7.0, 13.0), 'PO4': (1.50, 3.20), 'pH':
(5.4, 6.3)}
            ]

            all_x, all_y, all_nh4, all_po4, all_ph, all_loc = [],
[], [], [], [], []
            for i, params in enumerate(cluster_params):
                n_in_group = group_size if i < 3 else n_points - 3
* group_size

                x_start = left + i * (right - left) / 4.5
                x_end = left + (i + 1) * (right - left) / 4.5
                y_start = bottom + i * (top - bottom) / 4.5
                y_end = bottom + (i + 1) * (top - bottom) / 4.5
                x_group = np.random.uniform(x_start, x_end,
n_in_group)
                y_group = np.random.uniform(y_start, y_end,
n_in_group)

                nh4_group = np.random.uniform(params['NH4'][0],
                params['NH4'][1], n_in_group)
                po4_group = np.random.uniform(params['PO4'][0],
                params['PO4'][1], n_in_group)
                ph_group = np.random.uniform(params['pH'][0],
                params['pH'][1], n_in_group)
                loc_group = np.random.choice(locations,
n_in_group)

                all_x.extend(x_group)
                all_y.extend(y_group)
                all_nh4.extend(nh4_group)
```

```
                all_po4.extend(po4_group)
                all_ph.extend(ph_group)
                all_loc.extend(loc_group)

            self.points_gdf = gpd.GeoDataFrame(
                data={
                        'id': range(1, n_points + 1),
                        'NH4_mgL': np.round(all_nh4, 3),
                        'PO4_mgL': np.round(all_po4, 3),
                        'pH': np.round(all_ph, 2),
                        'location': all_loc
                    },
                geometry=gpd.points_from_xy(all_x, all_y),
                crs=self.raster.crs
            )

            print(f"Создано {len(self.points_gdf)} точек с 4
кластерами:")
            print(f"NH4: {self.points_gdf['NH4_mgL'].min():.2f} -
{self.points_gdf['NH4_mgL'].max():.2f}")
            print(f"Группы по NH4:
{self.points_gdf['NH4_mgL'].round(1).value_counts().sort_index().v
alues}")

            os.makedirs(os.path.dirname(output_file),
exist_ok=True)
            self.points_gdf.to_file(output_file, driver='GeoJSON')
            print(f"Сохранено в файле: {output_file}")

            return self.points_gdf


        def normalize(self, band):
            low, high = np.percentile(band, (2, 98))
            if high <= low:
```

```
                return np.zeros_like(band)
            return np.clip((band - low) / (high - low), 0, 1)


        def plot_rgb_with_points(self, filename='rgb_points.png'):
            """RGB-снимок с точками проб, цвет точки зависит от
NH4"""

            if self.raster is None or self.bands is None or
self.points_gdf is None:
                raise ValueError("Сначала загрузите снимок и
создайте точки проб")

            r, g, b = self.bands[0], self.bands[1], self.bands[2]
            rgb = np.dstack((self.normalize(r), self.normalize(g),
self.normalize(b)))

            fig, ax = plt.subplots(figsize=(10, 10))
            ax.imshow(rgb)

            points_in_image = 0
            for idx, row in self.points_gdf.iterrows():
                row_idx, col = self.raster.index(row.geometry.x,
row.geometry.y)
                if 0 <= col < rgb.shape[1] and 0 <= row_idx <
rgb.shape[0]:
                    points_in_image += 1
                    color = 'red' if row['NH4_mgL'] > 1.0 else
'orange' if row['NH4_mgL'] > 0.5 else 'green'
                    ax.scatter(col, row_idx, c=color, s=50,
alpha=0.6)

            ax.axis('off')
            ax.set_title(f'Точки проб ({points_in_image})')
```

```
            plt.tight_layout()
            plt.savefig(self.output_dir + filename, dpi=150)
            plt.show()


        def plot_index(self, name, cmap, vmin, vmax, title,
filename=None):
            """Карта одного спектрального индекса"""

            if name not in self.indices:
                raise ValueError(f"Индекс {name} не рассчитан")

            fig, ax = plt.subplots(figsize=(10, 10))
            im = ax.imshow(self.indices[name], cmap=cmap,
vmin=vmin, vmax=vmax)
            ax.set_title(title)
            plt.colorbar(im, ax=ax, fraction=0.03, pad=0.02)
            ax.axis('off')

            filename = filename or f'{name.lower()}.png'
            plt.tight_layout()
            plt.savefig(self.output_dir + filename, dpi=150)
            plt.show()


        def plot_nh4_histogram(self, filename='nh4_hist.png'):
            """Гистограмма концентрации NH4"""

            if self.points_gdf is None:
                raise ValueError("Сначала создайте точки проб")

            fig, ax = plt.subplots(figsize=(8, 5))
            ax.hist(self.points_gdf['NH4_mgL'], bins=8, alpha=0.7,
color='steelblue')
```

```
            ax.axvline(x=self.points_gdf['NH4_mgL'].mean(),
color='red', linestyle='--', label='Среднее')
            ax.set_title('Распределение NH4')
            ax.set_xlabel('NH4, мг/л')
            ax.set_ylabel('Количество проб')
            ax.legend()

            plt.tight_layout()
            plt.savefig(self.output_dir + filename, dpi=150)
            plt.show()


        def plot_locations(self, filename='locations.png'):
            """Распределение проб по локациям"""

            if self.points_gdf is None:
                raise ValueError("Сначала создайте точки проб")

            loc_counts                                          =
self.points_gdf['location'].value_counts()

            fig, ax = plt.subplots(figsize=(8, 5))
            ax.bar(loc_counts.index.astype(str).tolist(),
loc_counts.to_numpy(), alpha=0.7, color='steelblue')
            ax.set_title('Распределение проб по локациям')
            ax.set_ylabel('Количество проб')
            ax.tick_params(axis='x', rotation=45)

            plt.tight_layout()
            plt.savefig(self.output_dir + filename, dpi=150)
            plt.show()


        def plot_with_points(self):
            """Все графики по очереди, каждый в отдельном окне"""
```

```
            if not self.indices or self.points_gdf is None:
                raise ValueError("Сначала рассчитайте индексы и
создайте точки проб")

            self.plot_rgb_with_points()
            self.plot_index('NDVI', 'RdYlGn', -0.2, 0.8, 'NDVI
(растительность)')
            self.plot_index('NDWI', 'Blues', -0.5, 0.5, 'NDWI
(водные объекты)')
            self.plot_index('NDBI', 'Reds', -0.5, 0.5, 'NDBI
(застройка)')
            self.plot_nh4_histogram()
            self.plot_locations()


        def find_optimal_eps_auto(self, X_scaled, min_samples=4):
            """Вычисление оптимального eps"""

            neighbors = NearestNeighbors(n_neighbors=min_samples)
            distances,                          _                   =
neighbors.fit(X_scaled).kneighbors(X_scaled)
            distances = np.sort(distances[:, min_samples - 1])

            plt.figure(figsize=(12, 5))
            plt.subplot(1, 2, 1)
            plt.plot(distances)
            plt.title(f'k-distance график (k={min_samples})')
            plt.xlabel('Точки (отсортированы)')
            plt.ylabel(f'{min_samples}-е расстояние')
            plt.grid(True, alpha=0.3)

            try:
                kneedle    =    KneeLocator(range(len(distances)),
distances, curve='convex',
```

```
                                        direction='increasing',
S=1.0)
                if kneedle.knee is not None and kneedle.knee <
len(distances) * 0.8:
                    auto_eps = distances[kneedle.knee]
                else:
                    auto_eps = np.percentile(distances, 40)
            except (ImportError, Exception):
                print("KneeLocator     недоступен,     используем
перцентиль")
                auto_eps = np.percentile(distances, 40)

            best_eps = auto_eps
            best_score = -1
            eps_candidates = np.linspace(auto_eps * 0.6, auto_eps
* 1.8, 7)

            plt.subplot(1, 2, 2)
            silhouette_scores = []

            for eps_test in eps_candidates:
                try:
                    dbscan_test       =       DBSCAN(eps=eps_test,
min_samples=min_samples)
                    labels_test                                    =
dbscan_test.fit_predict(X_scaled)
                    n_clusters_test = len(set(labels_test)) - (1 if
-1 in labels_test else 0)

                    if n_clusters_test >= 2:
                        score     =     silhouette_score(X_scaled,
labels_test)
                        silhouette_scores.append(score)
                    else:
                        silhouette_scores.append(-1)
```

```
                except:
                    silhouette_scores.append(-1)

            valid_indices = [i for i, s in
enumerate(silhouette_scores) if s > -1]
            if valid_indices:
                best_idx = max(valid_indices, key=lambda i:
                silhouette_scores[i])
                best_eps = eps_candidates[best_idx]
                best_score = silhouette_scores[best_idx]

            print("\nМассив значений eps:")
            print(eps_candidates)
            print("Массив silhouette scores:")
            print(np.array(silhouette_scores))

            for i, eps_test in enumerate(eps_candidates):
                plt.scatter(eps_test, silhouette_scores[i],
                            s=120, c='green' if i == np.argmax([s
for s in silhouette_scores if s > -1]) else 'blue',
                            label=f'eps={eps_test:.3f}' if i < 3
else "")
            plt.axvline(best_eps, color='red', linestyle='--',
linewidth=2, label=f'Лучший: {best_eps:.3f}
(score={best_score:.3f})')
            plt.xlabel('eps')
            plt.ylabel('Silhouette Score')
            plt.title('Автоподбор eps')
            plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
            plt.grid(True, alpha=0.3)
            plt.tight_layout()
            plt.savefig(self.output_dir + 'clustering', dpi=150)
            plt.show()

            print(f"Найден eps: {best_eps:.3f}")
```

```
            print(f"Silhouette: {best_score:.3f}")
            print(f"Диапазон расстояний: {distances[5]:.3f} -
{np.percentile(distances, 75):.3f}")

            return best_eps


        def most_polluted_cluster(self, points_gdf, clusters):
            """Кластер с наибольшей средней концентрацией NH4 (без
шума)"""

            nh4 = pd.Series(points_gdf['NH4_mgL'].to_numpy(),
index=np.asarray(clusters))
            nh4 = nh4[nh4.index != -1]
            if nh4.empty:
                return None

            return nh4.groupby(level=0).mean().idxmax()


        def ndvi_near_water(self, x, y, radius_m=100):
            """NDVI в точке и наличие воды (NDWI > 0) в радиусе"""

            if self.raster is None or 'NDVI' not in self.indices or
'NDWI' not in self.indices:
                return None, False

            row, col = self.raster.index(x, y)
            height, width = self.indices['NDVI'].shape
            if not (0 <= row < height and 0 <= col < width):
                return None, False

            r = max(1, int(radius_m / self.raster.res[0]))
            window = self.indices['NDWI'][max(0, row - r):row + r
+ 1, max(0, col - r):col + r + 1]
```

```
            return     float(self.indices['NDVI'][row,     col]),
bool((window > 0).any())


        def get_recommendations(self, cluster, high_cluster, x, y):
            """Рекомендации для всплывающего окна точки"""

            recommendations = []
            if   high_cluster   is   not   None   and   cluster   ==
high_cluster:
                recommendations.append('Рекомендуется        отбор
повторной пробы')

            if cluster == -1:
                recommendations.append('Проверить      координаты:
возможна ошибка GPS')

            ndvi, near_water = self.ndvi_near_water(x, y)
            if ndvi is not None and ndvi < 0.2 and near_water:
                recommendations.append('Возможен   сброс   сточных
вод')

            return recommendations


        def     visualize_clusters_map(self,     points_gdf=None,
clusters=None, filename='clusters_map.html'):
            """Визуализация карты с кластерами проб"""

            if points_gdf is None:
                points_gdf = self.points_gdf
            if clusters is None:
                clusters = self.clusters
```

```
            if points_gdf is None or clusters is None:
                raise ValueError("Нет данных для визуализации
кластеров")

            os.makedirs(os.path.dirname(filename) if
            os.path.dirname(filename) else '.', exist_ok=True)
            print("\nКонвертация координат для отображения на
карте...")
            print(f"Исходная CRS точек: {getattr(points_gdf,
'crs', 'Не установлена')}")

            points_wgs84 = points_gdf.copy()

            current_crs = points_gdf.crs
            if current_crs and str(current_crs) != 'EPSG:4326':
                try:
                    points_wgs84 = points_gdf.to_crs('EPSG:4326')
                    print(f"Конвертировано в: {points_wgs84.crs}")
                except Exception as e:
                    print(f"Ошибка конвертации: {e}")
                    print("Пробуем принудительно установить
CRS...")
                    points_wgs84 = points_gdf.copy()
                    points_wgs84.crs = 'EPSG:4326'

            print(f"\nДиапазон координат после конвертации:")
            print(f"                Долгота                (X):
{points_wgs84.geometry.x.min():.6f}                              -
{points_wgs84.geometry.x.max():.6f}")
            print(f"                 Широта                (Y):
{points_wgs84.geometry.y.min():.6f}                              -
{points_wgs84.geometry.y.max():.6f}")

            center_lat = points_wgs84.geometry.y.mean()
            center_lon = points_wgs84.geometry.x.mean()
```

```
            print(f"Центр карты: {center_lat:.6f}, {center_lon:.6f}")

            m = folium.Map(location=[center_lat, center_lon], zoom_start=12)

            colors = ['blue', 'red', 'green', 'orange', 'purple', 'gray', 'brown', 'lightgray']

            high_cluster = self.most_polluted_cluster(points_gdf, clusters)

            points_added = 0
            for idx, row in points_wgs84.iterrows():
                cluster = clusters[idx]
                color = 'pink' if cluster == -1 else colors[cluster % len(colors)]

                radius = 5 + row['NH4_mgL'] * 3

                geom = points_gdf.loc[idx].geometry
                recommendations = self.get_recommendations(cluster, high_cluster, geom.x, geom.y)
                recommendations_html = ''.join(f'- {r}<br>' for r in recommendations)

                popup_text = f"""
                    <b>Точка {row.get('id', idx)}</b><br>
                    NH₄: {row['NH4_mgL']:.2f} мг/л<br>
                    PO₄: {row['PO4_mgL']:.2f} мг/л<br>
                    pH: {row['pH']:.2f}<br>
                    Кластер: {cluster}<br>
                    Локация: {row.get('location', 'Не указана')}<br>
```

```
                    {'<b>Рекомендации:</b><br>'                    +
recommendations_html if recommendations else ''}
                """

                folium.CircleMarker(
                    location=[row.geometry.y, row.geometry.x],
                    radius=radius,
                    popup=folium.Popup(popup_text, max_width=300),
                    color=color,
                    fill=True,
                    fillOpacity=0.7,
                    weight=2
                ).add_to(m)
                points_added += 1

            legend_html = '''
            <div style="position: fixed; bottom: 50px; left: 50px;
                z-index: 1000; background-color: white; padding:
10px;
                border-radius: 5px; border: 2px solid grey; font-
size:
                12px;">
                    <b>Легенда</b><br>
                    <span  style="color:  pink;">●</span>  Шум  (-
1)<br>
                    <span  style="color:  blue;">●</span>  Кластер
0<br>
                    <span  style="color:  red;">●</span>  Кластер
1<br>
                    <span  style="color:  green;">●</span>  Кластер
2<br>
                    <i>Размер = f(NH₄)</i>
            </div>
            '''
```

```
m.get_root().html.add_child(folium.Element(legend_html))

            folium.TileLayer(

tiles='https://server.arcgisonline.com/ArcGIS/rest/services/World_
Imagery/MapServer/tile/{z}/{y}/{x}',
                attr='Esri',
                name='Спутник'
            ).add_to(m)
            folium.LayerControl().add_to(m)

            m.save(filename)
            print(f"\nКарта кластеров сохранена: {filename}")
            print(f"Добавлено точек на карту: {points_added} из
{len(points_wgs84)}")

            webbrowser.open('file://'                              +
os.path.realpath(filename))

            return m


        def analyze_ndvi_anomalies(self, n_years=10, seed=42):
            """
            Анализ NDVI: STL + поиск аномалий
            1. Создает временной ряд по паттернам MODIS леса
            2. Добавляет случайные аномалии (засухи/вырубки) - для
            иллюстрации
            3. STL-декомпозиция - остатки - выявление аномалий ±2σ
            """
            np.random.seed(seed)
            random.seed(seed)
```

```
            dates = pd.date_range(start='2015-01-01',
periods=n_years * 12, freq='ME')
            months = dates.month
            seasonal_base = np.array([0.25, 0.28, 0.35, 0.45, 0.55,
0.70,
                                      0.78,  0.75,  0.65,  0.50,
0.35, 0.28])
            seasonal = seasonal_base[months - 1]

            trend = np.linspace(0, -0.12, n_years * 12)

            noise = np.random.normal(0, 0.015, len(dates))

            anomalies = np.zeros(len(dates))
            anomaly_years = random.sample(range(2015, 2015 +
n_years), min(2, n_years // 3))
            for year in anomaly_years:
                start_month = random.randint(4, 8)
                months_in_year = random.randint(4, 8)
                idx_start = (year - 2015) * 12 + start_month
                anomalies[idx_start:idx_start + months_in_year] -=
np.random.uniform(0.15, 0.30)

            ndvi = np.clip(trend + seasonal + noise + anomalies, 0,
1)
            ts_df = pd.DataFrame({'date': dates, 'ndvi':
ndvi}).set_index('date')
            ts_df.to_csv(os.path.join(self.output_dir,
'realistic_ndvi.csv'))

            print(f"Реалистичный NDVI {ts_df.index.min().year}-
{ts_df.index.max().year}")
            print(f"Случайные аномалии добавлены в годы:
{anomaly_years}")
```

```
            stl = STL(ts_df.ndvi, period=12, robust=True)
            result = stl.fit()

            resid_mean,    resid_std    =    result.resid.mean(),
result.resid.std()
            upper, lower = resid_mean + 2 * resid_std, resid_mean
- 2 * resid_std

            anomalies_mask = (result.resid > upper) | (result.resid
< lower)

            self.plot_stl_component(result.observed,  'Наблюдаемый
NDVI', 'NDVI', 'ndvi_stl_observed.png')
            self.plot_stl_component(result.trend,  'Тренд  NDVI',
'Тренд', 'ndvi_stl_trend.png')
            self.plot_stl_component(result.seasonal,  'Сезонность
NDVI', 'Сезонность', 'ndvi_stl_seasonal.png')
            self.plot_stl_residuals(result.resid,  lower,  upper,
anomalies_mask)
            self.plot_ndvi_with_anomalies(ts_df.ndvi,
anomalies_mask)

            print("\nАномалии обнаружены автоматически")
            print("=" * 70)
            anomaly_df = ts_df[anomalies_mask].copy()
            anomaly_df['resid'] = result.resid[anomalies_mask]
            anomaly_df['zscore'] = (anomaly_df.resid - resid_mean)
/ resid_std
            print(anomaly_df.round(3))

            print("\nИнтерпретация:")
            for idx in anomaly_df.index:
                z = anomaly_df.loc[idx, 'zscore']
                year = idx.year
                if z < -2.5:
```

```
                    event = "Сильная засуха/ вырубка"
                elif z < -1.8:
                    event = "Умеренная засуха/болезнь"
                else:
                    event = "Аномалия (анализ спутников/полей)"
                print(f"  {idx.strftime('%Y-%m')}:  z={z:.2f}  →
{event}")

            self.results['analyze_ndvi_anomalies'] = {
                'df': ts_df, 'stl': result, 'anomalies_mask':
                anomalies_mask,
                'anomaly_years_true': anomaly_years,
                'anomaly_years_detected':
anomaly_df.index.year.unique()
            }
            return ts_df, anomaly_df


        def  plot_stl_component(self,  series,  title,  ylabel,
filename):
            """Одна компонента STL-разложения: наблюдаемый ряд,
тренд или сезонность"""

            fig, ax = plt.subplots(figsize=(15, 4))
            ax.plot(series)
            ax.set_title(title)
            ax.set_xlabel('Дата')
            ax.set_ylabel(ylabel)
            ax.grid(True, alpha=0.3)

            plt.tight_layout()
            plt.savefig(os.path.join(self.output_dir,  filename),
dpi=150, bbox_inches='tight')
            plt.show()
```

```
        def plot_stl_residuals(self, resid, lower, upper,
anomalies_mask, filename='ndvi_stl_resid.png'):
            """Остатки STL с коридором ±2σ и найденными
аномалиями"""

            fig, ax = plt.subplots(figsize=(15, 4))
            ax.plot(resid)
            ax.fill_between(resid.index,       lower,       upper,
color='gray', alpha=0.2, label='Коридор ±2σ')
            ax.scatter(resid.index[anomalies_mask],
resid[anomalies_mask],
                       color='red',  s=40,  label='Аномалии  ±2σ',
zorder=5)
            ax.set_title('Остатки STL-разложения')
            ax.set_xlabel('Дата')
            ax.set_ylabel('Остатки')
            ax.legend()
            ax.grid(True, alpha=0.3)

            plt.tight_layout()
            plt.savefig(os.path.join(self.output_dir,   filename),
dpi=150, bbox_inches='tight')
            plt.show()


        def  plot_ndvi_with_anomalies(self,  ndvi,  anomalies_mask,
filename='ndvi_anomalies.png'):
            """Исходный ряд NDVI с отмеченными аномалиями"""

            fig, ax = plt.subplots(figsize=(15, 4))
            ax.plot(ndvi, label='NDVI')
            ax.scatter(ndvi.index[anomalies_mask],
ndvi[anomalies_mask],
```

```
                        color='red',    s=60,    label='Обнаруженные
аномалии', zorder=5)
            ax.set_title('NDVI с найденными аномалиями')
            ax.set_xlabel('Дата')
            ax.set_ylabel('NDVI')
            ax.legend()
            ax.grid(True, alpha=0.3)

            plt.tight_layout()
            plt.savefig(os.path.join(self.output_dir,   filename),
dpi=150, bbox_inches='tight')
            plt.show()


        def       plot_ndvi_histograms(self,       ndvi_by_label,
filename='ndvi_hist.png'):
            """Гистограммы NDVI нескольких снимков на одном
графике"""

            fig, ax = plt.subplots(figsize=(10, 5))
            colors = ['green', 'red']
            for (label, ndvi), color in zip(ndvi_by_label.items(),
colors):
                median = float(np.median(ndvi))
                ax.hist(ndvi.ravel(),  bins=100,  range=(-1,  1),
density=True,
                        alpha=0.5, color=color, label=label)
                ax.axvline(median,  color=color,  linestyle='--',
label=f'Медиана: {median:.2f}')

            ax.set_title('Распределение NDVI')
            ax.set_xlabel('NDVI')
            ax.set_ylabel('Плотность')
            ax.legend()
            ax.grid(True, alpha=0.3)
```

```
        plt.tight_layout()
        plt.savefig(self.output_dir + filename, dpi=150)
        plt.show()
```